IE 643: Deep Learning - Theory and Practice **July-Dec 2026**

Project Report: Making Large Models Draw Topological Graphs

Team Name: Pensieve *Team Members: Adarsh A Rao (24B2132)*

Abstract

We ask whether a large language model can read the PyTorch source of a neural network and write down its forward-pass graph without running the code. Because `torch.fx` traces the true graph, every answer can be graded exactly by graph isomorphism. We build a generator of 1,600 programs in four tiers of increasing hidden structure, and test two approaches. Prompted zero-shot, accuracy grows with model size, from 5% of graphs with the right topology for Qwen2.5-Coder 1.5B to 60% for the 14B and 93% for Gemma 4 31B; branching code is the last skill to appear. Fine-tuning the 1.5B model with LoRA for 97 minutes on one free GPU raises it to 40 of 40 test graphs exactly right, and 20 of 20 on fresh programs, with no data leak. Three probes outside the generator then show what was learned: on real torchvision code the tuned model gets none of 21 graphs right and draws most as a straight chain; when an edit changes the wiring, it returns the graph of the unedited program 68% of the time; and it deletes a branch even when its flag is `True`, because every flag in training was `False`. A perfect in-distribution score and a clean leak audit did not show understanding; behavioural probes with exact ground truth did. We also provide a test interface that draws the model's graph beside the true one for any pasted, uploaded or generated program.

1 Introduction

The allotted task was to study whether LLMs could draw and understand topological graphs. And the specific task taken up was that of a familiar one - to **trace** and generate the forward-pass graph of a deep neural network's `forward()` method (specifically pytorch). This tracing may not be as easy as simply following the code line by line, because architectures that are relatively easy to understand for us, may not be so for LLMs, even simple loops, conditional statements are out of the ordinary for a rookie LLM. But ofcourse, with the whole premise of Deep Learning being "Deep", as the architectures become ridiculously more complex, it's almost impossible for a human to trace the flow of data, from the input through its processing to it's final form of some kind of prediction. And so we ask if these LLMs can be taught to understand such large structures. An LLM is a natural choice for such a problem because code is text and text is what we feed these LLMs.

Here are some of the **Research Questions** we not only started off with but also realised through the project:

1. How does accuracy fall as the code hides more of the structure? And as the graph grows?
2. Is the hard part *reading the code* or *building the graph*?
3. Prompting vs fine-tuning vs from scratch: best accuracy for the compute spent?
4. Do models learn the rule, or memorise the generator's habits?
5. Does a model track dataflow, or match patterns? Controlled edits that keep the graph should not change its answer; edits that change the graph should change it correctly.

6. Can a fine-tuned model handle a construct it never saw in training?

Some things that we ended up building through the project that might be of some importance (for the community, but a greater learning outcome for me):

1. **An exactly graded benchmark for code → graph**
   - A generator with 4 tiers of difficulty, by how much the code hides the graph: flat, loops, branches, blocks. 1,600 programs.
   - Ground truth from torch.fx, graded by graph isomorphism (a schema).
2. **An evaluation toolkit that says why an answer is wrong, not just whether**
   - A validator that checks a graph without the answer: cycles, reachability, hierarchy, arity, shapes.
   - Graded strictness levels: exact, topology, right graph.
   - Plus node and edge F1 and connectivity preservation.
   - A rule-based error classifier that puts every miss into one category.
3. **Applying prompting ideas, compared on multiple models**
   - Chain-of-simulation: a prompt that traces which node each variable holds. It was best on the two hardest tiers.
   - The read/wire decomposition (nodes only vs edges only). It showed the two skills fail independently, both on branching code.
4. **A fine-tuned small model that solves the in-distribution (important caveat) task**
5. **Behavioural probes of "understanding"**:

   Spoiler: Real code, controlled edits and a held-out construct show the tuned model learned the generator's shortcuts

2 Project Workflow

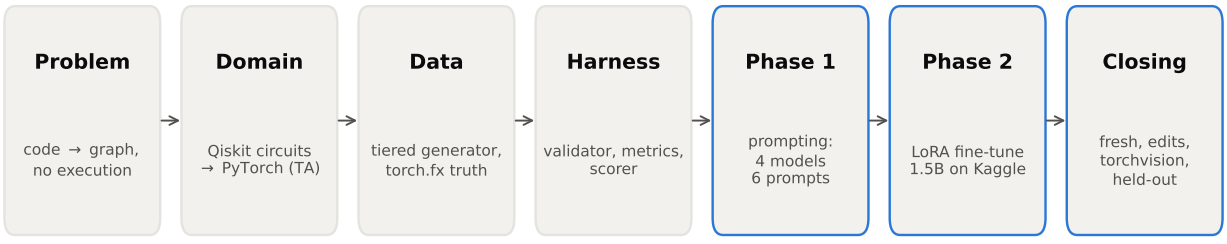


Figure 1: Project workflow, from the problem statement to the closing experiments.

How we travelled through this project:

1. Plan of action

2. Mid-term Prep Review
    - Initially started off with a qiskit quantum circuit DAG graphArt idea
    - Built the script generation, and evaluation pipeline for these DAGs
3. Correction after the prep review
    - Shifted focus to the required Deep Neural Network forward pass graph building
    - Data generation and evaluation had to be rebuilt (viewpoint changes on the already built ones)
4. Phase 1 prompting on various viable models
5. Phase 2 LoRA fine tuning
6. Imporant checks and out of generated - torchvision testing and realizations
7. Putting it all together in the requested format

3 Literature Survey

LLMs reading and writing graphs as text - [3], Accuracy depends on how a graph is encoded as text; LLMs struggle once graph tasks need multi-step reasoning You choose one fixed text encoding and grade it exactly; the graph has to come out of code, not out of a description

Throughout the project, we have mostly experimented with the natural direciton of the problem and honestly, did not rely and take much inspiration from established literature. This could be one of our many wrong choices but this decision was not made out of indifference to the problem.

4 Proposed Approach or Approaches

4.1 Work done before prep-presentation review

Before the prep-presentation (midterm) review;

- A complete pipeline for generating and evaluating a tiered qiskit circuit generator.
- The ground truth was pulled from qiskit's built in `<circuit_to_dag>` function.
- As mentioned the generator created circuits with multiple levels of "difficulty" - basically hiding more and more code with the higher levels.
- A common schema was developed for isomorphing, the LLM's generated graph to qiskit's standards.
- Validators were needed to confirm that the LLM produced a real / viable graph.
- The metrics (valid / exact / node F1 / edge F1) were decided upon and the complete prompting pipeline was built with standard prompting techniques.

Nothing else was run or built.

4.2 Work done after prep-presentation review

After the review; task reformulated as:

- input : Python source defining an nn.Module + example_input; and output : graph_text
- Ground truth: use torch.fx symbolic_trace + ShapeProp;
- Validation: Checks for desired replies: acyclic, one output, reachability, arities, module-path tree, shape consistency
- Metrics (graphart/metrics.py): valid; exact = labelled isomorphism; topo = exact ignoring module parameters and function-vs-method spelling; node/edge F1 on label multisets; shapes scored separately as a 'right graph'.
- Error classifier: for precise reasoning of the error / mistake
- Phase 1: Prompting : zero_shot, few_shot, cot, cos = chain-of-simulation (interpret forward() line by line, track which node each variable points to), nodes_only / edges_only (decomposition).
- Phase 2: LoRA fine-tuning; performed on the smallest Qwen2.5-Coder-1.5B-Instruct, decoder-only transformer, 1.56B params fusing Kaggle's free GPU access.

4.3 Test interface

- Multiple input methods: paste code, upload a .py file, pick one of 254 examples (40 test, 21 torchvision, 193 edits), or GENERATE a fresh program with the project's generator (tier + seed).
- Reference: torch.fx trace of the code
- Prediction: the fine-tuned 1.5B - (easiest on my cpus)
- Draw the final graphs and run the comparison, ouputs side by side graphs, 8 metrics etc.

5 Data set Details

Tier	programs	train	val	test	nodes min / mean / max	merges / graph
1 flat	400	320	40	40	7 / 10.1 / 14	0.00
2 loops	400	320	40	40	8 / 10.4 / 13	0.00
3 branches	400	320	40	40	8 / 12.0 / 16	2.02
4 blocks	400	320	40	40	12 / 15.1 / 25	1.88
all	1600	1280	160	160		

Table 1: The synthetic data set: four tiers by how much the code hides the graph. A merge is a node with more than one incoming value (residuals, concatenations).

- Fully synthetically generated using a builder script.
- 4 tiers by how much the code HIDES the graph: T1 flat, T2 loops/Sequential/ModuleList, T3 branches (residual, concat, dead branch, reused layer), T4 blocks (residual CNN, inception, transformer, U-net);

6 Experiments

The team should include all experiments, even those with negative results, and address all experiments that were performed.

- Hardware: laptop (Windows, Intel Iris Xe, CPU only) for local runs; Kaggle 2x NVIDIA T4 16 GB (training uses one); OpenRouter free tier (50 requests/day) for Gemma
- Models: Qwen2.5-Coder 1.5B / 7B / 14B (Ollama, 4-bit), Gemma 4 31B (OpenRouter), 1.5B fp16 (HF)
- Decoding: greedy / temperature 0; 1500-token cap (4096 for cot/cos, 3200 for torchvision); Ollama num_ctx 8192, think=false
- Training (Phase 2): HF Trainer, AdamW (fused, default betas), lr 2e-4, cosine schedule, 3% warmup, 2 epochs = 160 steps, effective batch 16 (1 x 16 accumulation), max length 2560, fp16, gradient checkpointing, seed 0, val loss every 40 steps, best checkpoint kept; 97.5 min val loss 0.0016 → 0.0003 → 0.00009 → 0.00009
- Negative results / problems (template asks for ALL experiments):
  - first reply failed to parse: prompts defined labels for 2 of 14 layer types → label table + topo
  - cos prompt lacked the Sequential rule → re-run; did not improve tier 2
  - free OpenRouter endpoint ∼1 reply / 15-20 min → moved to Kaggle
  - Gemma 4 "thinking" returned empty replies → think=false
  - two Kaggle fine-tuning failures: 35 examples > 2048 tokens; Kaggle's old torchao vs peft
- Speeds measured on T4: tuned 1.5B ∼24 tok/s, Qwen 7B ∼34, Qwen 14B ∼16; Gemma ∼5
- Closing runs (each capped at 2.5 h of Kaggle time): Qwen 14B zero-shot via Ollama on torchvision + edits; tuned 1.5B (HF, greedy) on torchvision + edits + rq6; held-out adapter trained WITHOUT any dead-branch program (1112 examples, ONE epoch = 70 steps, 43 min, same hyper-parameters otherwise) and run on rq6. No reply truncated (all have their edge list).
- think=false: matters only for models with a thinking mode (Gemma 4 returned empty replies, 288 s for 0 characters); Qwen2.5-Coder has none, so it changed nothing for any Qwen run

7 Results

Model (zero-shot)	T1	T2	T3	T4	topo	exact	right graph	reach F1	parsed
Qwen 1.5B	0.20	0.00	0.00	0.00	0.05	0.00	0.07	0.18	0.62
Qwen 7B	0.50	0.50	0.00	0.00	0.25	0.20	0.42	0.57	0.80
Qwen 14B	1.00	0.80	0.20	0.40	0.60	0.17	0.70	0.44	0.47
Gemma 4 31B	1.00	1.00	1.00	0.70	0.93	0.75	0.93	0.93	0.93

Table 2: Zero-shot accuracy by model size on the 40 stratified test programs (10 per tier). Tier columns and *topo* count graphs with the right topology; *exact* is strict; *right graph* also forgives spelling that does not change the computation; *reach F1* measures connectivity preservation (shared ancestor–descendant pairs).

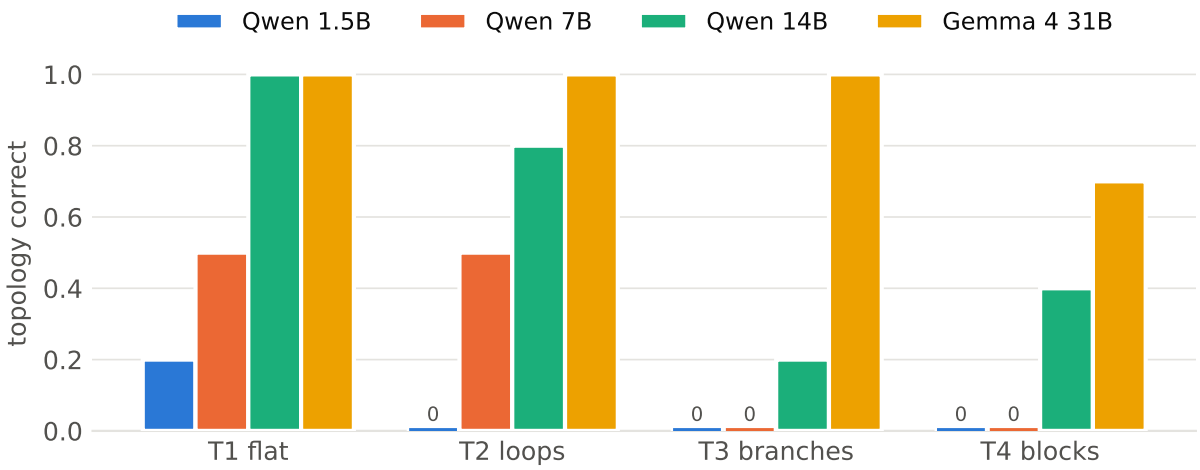


Figure 2: Zero-shot: share of the 40 test programs with the correct topology, by tier and model size.

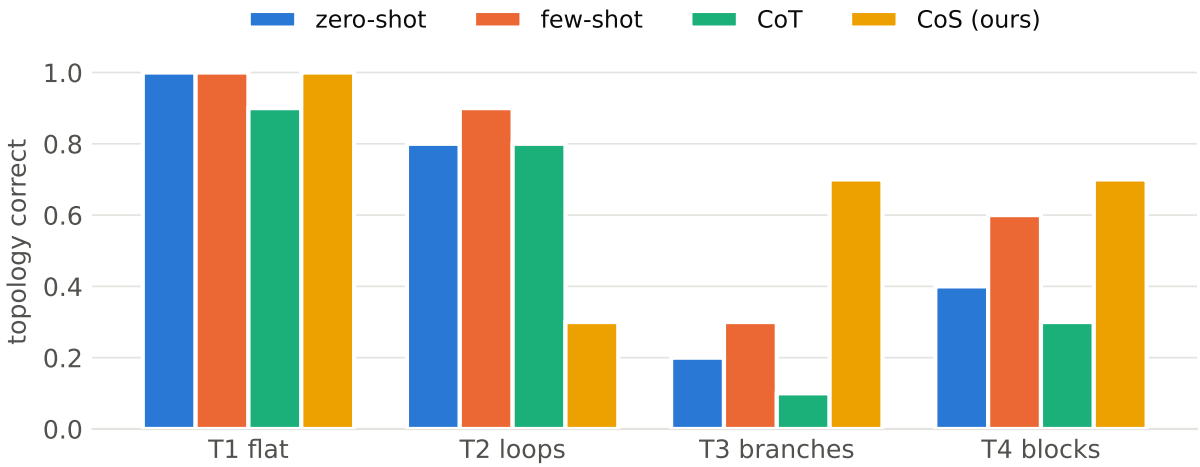


Figure 3: Prompting strategies on Qwen2.5-Coder 14B, by tier (topology correct).

Prompt (Qwen 14B)	T1	T2	T3	T4	topo	exact	right graph	parsed
zero-shot	1.00	0.80	0.20	0.40	0.60	0.17	0.70	0.47
few-shot	1.00	0.90	0.30	0.60	0.70	0.47	0.78	0.90
CoT	0.90	0.80	0.10	0.30	0.53	0.15	0.60	0.55
CoS (ours)	1.00	0.30	0.70	0.70	0.68	0.30	0.68	0.62
nodes only	1.00	0.80	0.40	0.60	0.70	0.17	–	0.47
edges only	1.00	1.00	0.40	0.60	0.75	0.75	–	1.00
full task	1.00	0.80	0.20	0.40	0.60	0.17	0.70	0.47

Table 3: Prompting strategies and the read/wire decomposition on Qwen2.5-Coder 14B, same 40 programs. *right graph* applies to whole-graph replies only.

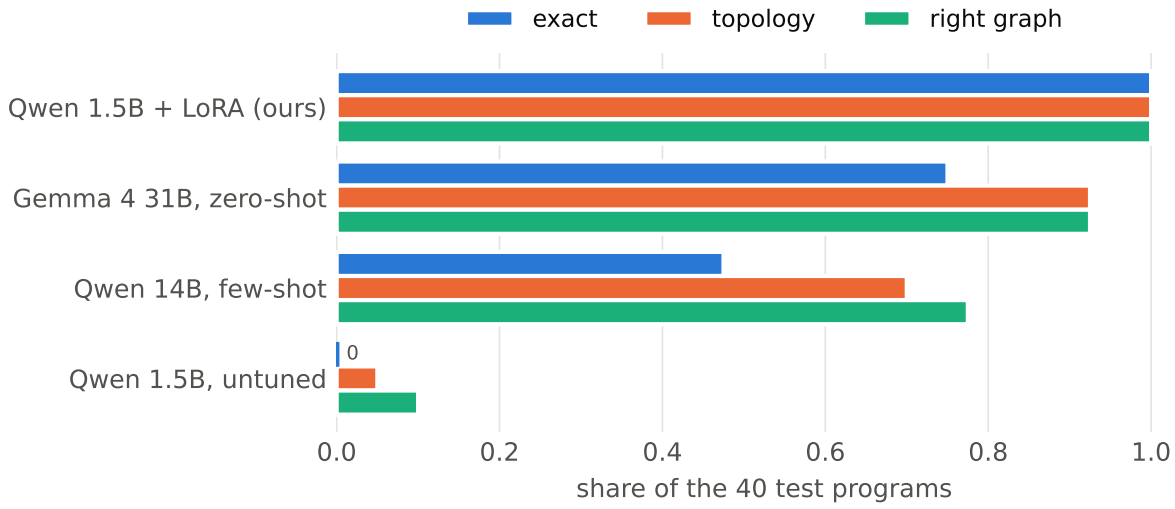


Figure 4: The LoRA-tuned 1.5B against its untuned base and the best prompted models.

Model	exact	topo	right graph	reach F1	parsed
Qwen 1.5B, untuned	0.00	0.05	0.10	0.32	0.75
Qwen 14B, few-shot	0.47	0.70	0.78	0.88	0.90
Gemma 4 31B, zero-shot	0.75	0.93	0.93	0.93	0.93
Qwen 1.5B + LoRA (ours)	1.00	1.00	1.00	1.00	1.00

Table 4: Fine-tuning against prompting, same 40 test programs and the same zero-shot prompt for the two 1.5B rows. All on our generator's programs; Table 5 has real code.

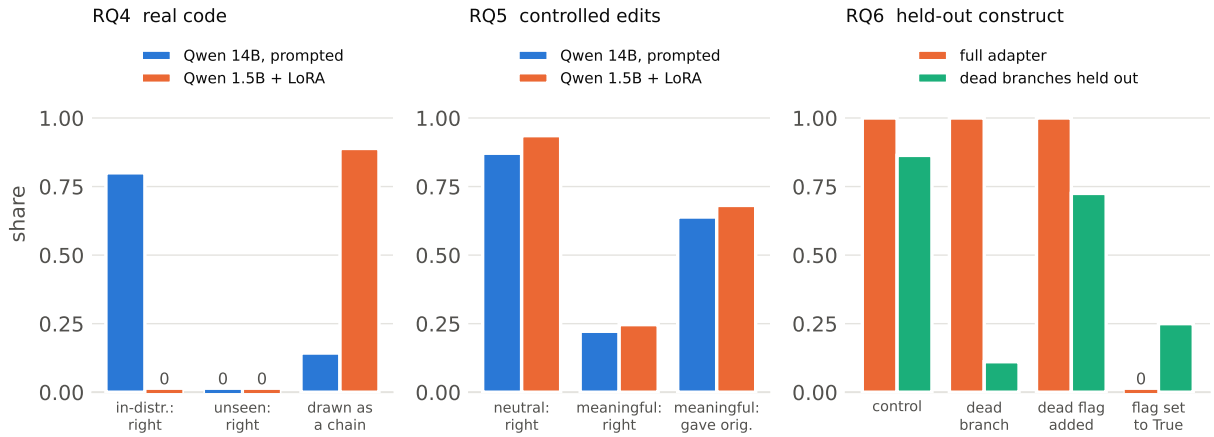


Figure 5: Closing experiments. RQ4: torchvision, right graph on in-distribution and unseen-construct samples, and the share of readable replies drawn as one straight chain. RQ5: controlled edits on programs each model solved first. RQ6: the full adapter against one trained without dead branches.

- (RQ1)Zero-shot topology climbs with size: 1.5B: 0.05 7B: 0.25 14B: 0.60 Gemma 31B: 0.93

torchvision (21)	parsed	exact	topo	right	node F1	edge F1	reach F1	in-distr.	unseen	chain
Qwen 14B, prompted	14	1	1	4	0.31	0.20	0.21	4 / 5	0 / 16	2 / 14
Qwen 1.5B + LoRA	18	0	0	0	0.36	0.17	0.31	0 / 5	0 / 16	16 / 18

Table 5: RQ4, real code: counts over 21 torchvision samples. *In-distr.* (5 samples) have none of the constructs absent from all 1,280 training programs (own block classes, grouped convolutions, tuple kernels, un-shortened `F.` functions, `assert`); the rule was fixed before the tuned model's results were opened. *Right* is the right-graph level. *Chain*: readable replies with no branch at all; 4 of the 21 true graphs are chains.

edits	solved	neutral	meaningful	gave original	reroute	swap cat	drop call
Qwen 14B, prompted	24	61 / 70	8 / 36	23 / 36	0 / 14	0 / 4	8 / 18
Qwen 1.5B + LoRA	40	116 / 124	17 / 69	47 / 69	0 / 27	0 / 9	17 / 29

Table 6: RQ5, controlled edits of the test programs, counting only the programs each model got right before the edit (*solved*). Columns count right graphs, except *gave original*: meaningful edits answered with the graph of the unedited program.

rq6 probe (84)	control	dead branch	dead flag added	flag True	dead layer in
full adapter	22 / 22	18 / 18	40 / 40	0 / 4	0 / 58
dead branches held out	19 / 22	2 / 18	29 / 40	1 / 4	18 / 58

Table 7: RQ6, a held-out construct: the full adapter against one trained without any dead-branch program (1,112 examples, one epoch against two). Group columns count topology-correct graphs. All 168 flagged branches in the training data have the flag `False`.

- (RQ1)Chains come before branches. The 7B gets 0 on tiers 3–4; branching first appears at 14B (T3 0.20, T4 0.40); Gemma solves tier 3 completely (1.00).
- (RQ2) Decomposition: reading alone (nodes only): 0.70 wiring given nodes (edges only): 0.75 both at once: 0.60
- (RQ2) Prompts: CoT: 0.53, below zero-shot (0.60). Its replies are about 2.4× longer (median 1,652 vs 698 characters) and drift from the format more. Few-shot: best overall at 0.70, mostly by fixing format. Raw parse failures fall 21 → 4, exact rises 7 → 19. CoS: best on the hard tiers, but collapses on loops. Therefore, we infer that structured simulation helps where reasoning is the bottleneck; and giving examples clears out the format.
- (RQ3) Fine-tuning (1.87 percent): LoRA 1.00 on every metric for the worst performing 1.5B model. Found it a little suspicious at first, so audited it for any leakage of information, and so tested with 20 newly generated programs - still scored perfectly.
- Outside the Generators: THe real test and the real performance on real torchvision codes LoRA performs poorly, worse than the zero shot prompted 14B model; but a better Reach F1. So it's clear that LoRA fine tuned 1.5B simply memorised shortcuts for the house style generated code.

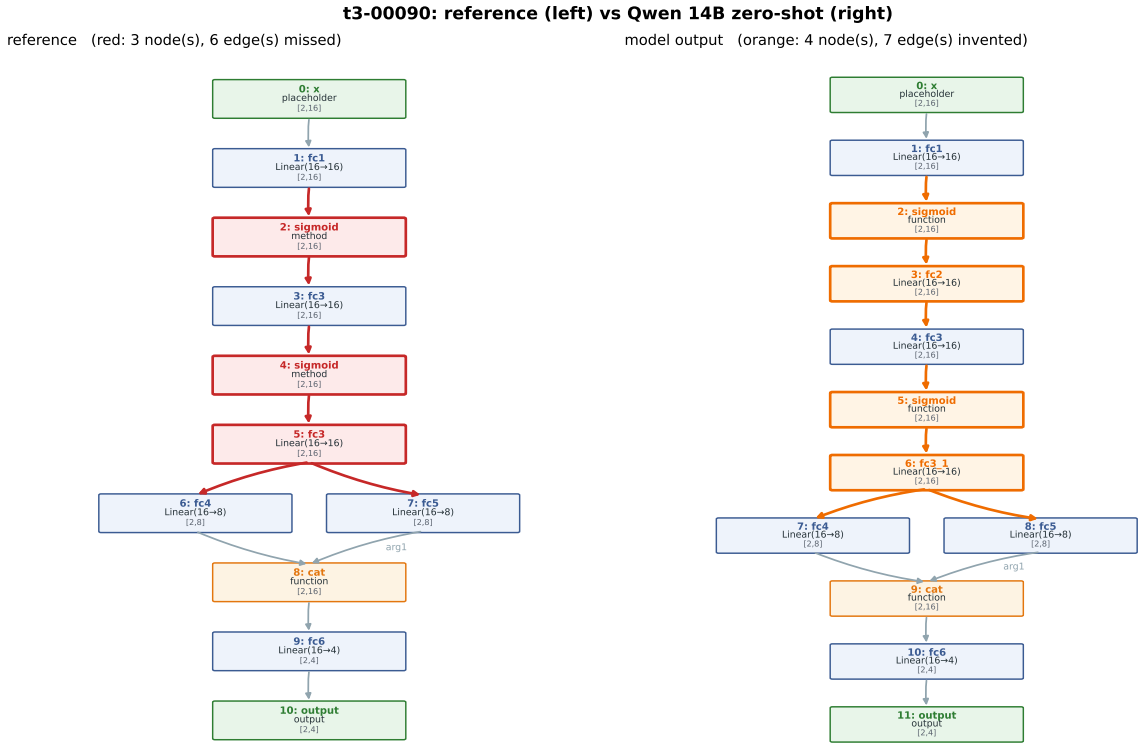


Figure 6: Qualitative: Qwen 14B zero-shot on a tier-3 test program. Red: in the reference, missed; orange: invented.

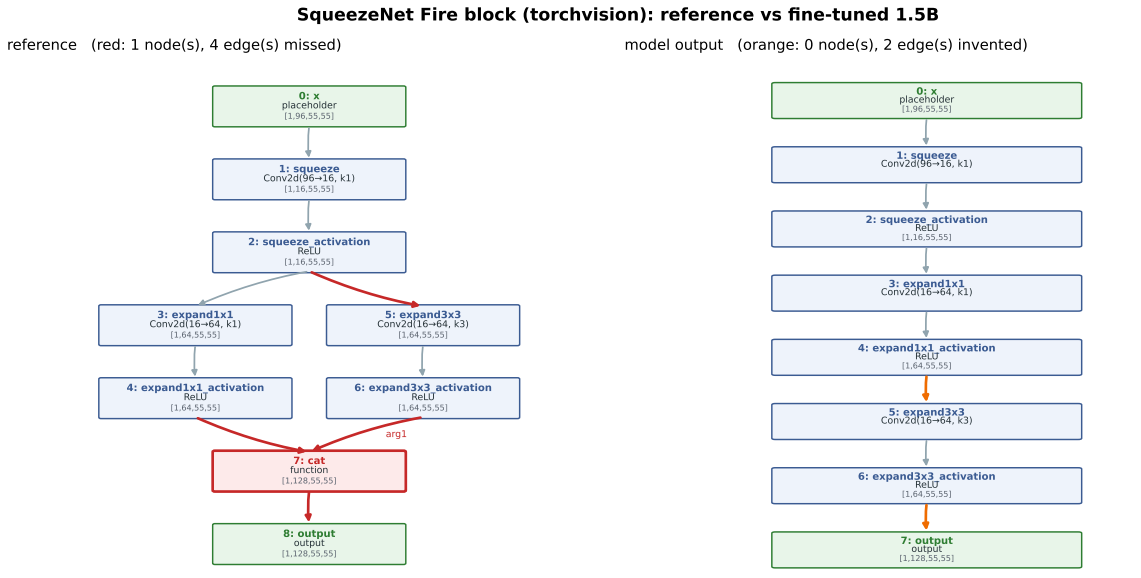


Figure 7: Qualitative: the fine-tuned 1.5B on torchvision's SqueezeNet Fire block draws a chain.

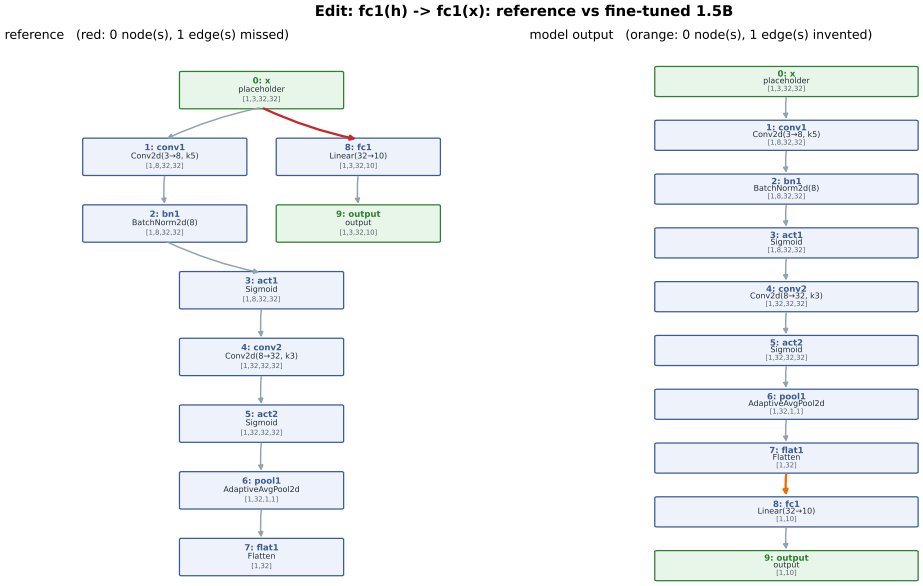


Figure 8: Qualitative: the fine-tuned 1.5B on a rerouted input answers with the unedited graph.

8 Plan for Novelty Assessment

The closing experiments did more than measure accuracy: they showed which shortcuts the fine-tuned model took, and each of them points to a change in the data. The plan for the novelty assessment is to remove those shortcuts and test whether the model then learns the rule.

1. **Remove the shortcuts from the generator (main idea).** Add flags that are sometimes `True`, varied names for the input, helper classes and helper functions of the kind torchvision uses, and layers outside the current menu such as grouped convolutions and non-square kernels. Retrain the same LoRA set-up and re-run the three probes. Success means the tuned model gets some of the torchvision graphs right (above 0 of 21), answers far fewer meaningful edits with the old graph (well below 68%), and keeps branches whose flag is `True`.

2. **Train on edit pairs.** Add training pairs that differ only in their wiring — the same program with one input rerouted — so that the answer cannot be produced from the familiar shape of the code. Test on the held-out edit set.

3. **Firmer numbers.** Repeat the main runs over several training seeds and on all 160 test programs, and run the strongest prompted model (Gemma 4 31B) on the edit set, to see whether the old-graph answers also appear at larger scale.

Each retraining takes about 100 minutes on a free Kaggle GPU, and the probes, scoring and error classification already exist, so the plan fits in the time before the assessment.

9 Conclusion

This project asked whether a language model can turn the source code of a PyTorch network into its forward-pass graph without running it, and whether it understands the structure it draws. Using `torch.fx` as an exact answer key, a tiered generator of programs, and a pipeline that grades replies by graph isomorphism, we compared prompting across four models and six prompts with LoRA fine-tuning of a small model.

Prompting improves with scale and with the right prompt: topology accuracy rises from 0.05 to 0.93 between 1.5B and 31B parameters, branching code is solved last, and simulating the code line by line helps on the hardest tiers while worked examples help most with the output format. Fine-tuning a 1.5B model is enough to solve the task on our generator's programs, including programs it has never seen.

The closing experiments qualify that result. Outside the generator the tuned model fails, and both it and the prompted 14B answer changed wiring with the graph the code used to have. The model learned to read code from one family, together with shortcuts that family allowed. The broader lesson is about evaluation: a perfect score on held-out data and a clean leak audit were not evidence of understanding, while small controlled probes with exact ground truth were. Removing those shortcuts from the data, and testing again, is the next step.

Code

The code, data generator, prompts, notebooks and results are available at `https://github.com/Rao-Adarsh/graphArt_LM`.

References

[1] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, et al. Language models are few-shot learners. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020. https://arxiv.org/abs/2005.14165.

[2] Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. QLoRA: Efficient finetuning of quantized LLMs. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. https://arxiv.org/abs/2305.14314.

[3] Bahare Fatemi, Jonathan Halcrow, and Bryan Perozzi. Talk like a graph: Encoding graphs for large language models. In *International Conference on Learning Representations (ICLR)*, 2024. https://arxiv.org/abs/2310.04560.

[4] Gemma Team, Aishwarya Kamath, Johan Ferret, Shreya Pathak, Nino Vieillard, et al. Gemma 3 technical report. *arXiv preprint arXiv:2503.19786*, 2025. https://arxiv.org/abs/2503.19786.

[5] Google DeepMind. Gemma 4 model family. https://ollama.com/library/gemma4. Accessed on: 26th October, 2026.

[6] Alex Gu, Baptiste Rozière, Hugh Leather, Armando Solar-Lezama, Gabriel Synnaeve, and Sida I. Wang. CRUXEval: A benchmark for code reasoning, understanding and execution. In *International Conference on Machine Learning (ICML)*, 2024. https://arxiv.org/abs/2401.03065.

[7] Aric A. Hagberg, Daniel A. Schult, and Pieter J. Swart. Exploring network structure, dynamics, and function using NetworkX. Proceedings of the 7th Python in Science Conference (SciPy 2008); https://networkx.org, 2008. Accessed on: 26th October, 2026.

[8] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations (ICLR)*, 2022. https://arxiv.org/abs/2106.09685.

[9] Binyuan Hui, Jian Yang, Zeyu Cui, Jiaxi Yang, Dayiheng Liu, Lei Zhang, et al. Qwen2.5-Coder technical report. *arXiv preprint arXiv:2409.12186*, 2024. https://arxiv.org/abs/2409.12186.

[10] Kaggle. Kaggle notebooks (free GPU sessions). https://www.kaggle.com/code. Accessed on: 26th October, 2026.

[11] Sourab Mangrulkar, Sylvain Gugger, Lysandre Debut, Younes Belkada, Sayak Paul, and Benjamin Bossan. PEFT: State-of-the-art parameter-efficient fine-tuning methods. https://github.com/huggingface/peft, 2022. Accessed on: 26th October, 2026.

[12] Maxwell Nye, Anders Johan Andreassen, Guy Gur-Ari, Henryk Michalewski, Jacob Austin, David Bieber, et al. Show your work: Scratchpads for intermediate computation with language models. *arXiv preprint arXiv:2112.00114*, 2021. https://arxiv.org/abs/2112.00114.

[13] Ollama. Ollama: run large language models locally. https://ollama.com. Accessed on: 26th October, 2026.

[14] OpenRouter. OpenRouter: a unified interface for LLMs. https://openrouter.ai. Accessed on: 26th October, 2026.

[15] James K. Reed, Zachary DeVito, Horace He, Ansley Ussery, and Jason Ansel. torch.fx: Practical program capture and transformation for deep learning in Python. In *Proceedings of Machine Learning and Systems (MLSys)*, 2022. https://arxiv.org/abs/2112.08429.

[16] TorchVision maintainers and contributors. TorchVision: PyTorch's computer vision library. `https://github.com/pytorch/vision`, 2016. Accessed on: 26th October, 2026.

[17] Heng Wang, Shangbin Feng, Tianxing He, Zhaoxuan Tan, Xiaochuang Han, and Yulia Tsvetkov. Can language models solve graph problems in natural language? In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. `https://arxiv.org/abs/2305.10037`.

[18] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022. `https://arxiv.org/abs/2201.11903`.

[19] Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, et al. Transformers: State-of-the-art natural language processing. In *Proceedings of EMNLP 2020: System Demonstrations*, 2020. `https://arxiv.org/abs/1910.03771`.

[20] Wojciech Zaremba and Ilya Sutskever. Learning to execute. *arXiv preprint arXiv:1410.4615*, 2014. `https://arxiv.org/abs/1410.4615`.